tf.compat.v1.data.experimental.CsvDataset Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/data/experimental/CsvDataset

> *A Dataset comprising lines from one or more CSV files.*

Function Signatures

```
tf.compat.v1.data.experimental.CsvDataset( filenames,
record_defaults, compression_type=None, buffer_size=None,
header=False, field_delim=',', use_quote_delim=True,
na_value='', select_cols=None, exclude_cols=None )
```

Code Examples

```
tf.compat.v1.data.experimental.CsvDataset(
filenames,
record_defaults,
compression_type=None,
buffer_size=None,
header=False,
field_delim=',',
use_quote_delim=True,
na_value='',
select_cols=None,
exclude_cols=None
)
```

```
tf.compat.v1.data.experimental.CsvDataset(
filenames,
record_defaults,
compression_type=None,
buffer_size=None,
header=False,
field_delim=',',
use_quote_delim=True,
na_value='',
select_cols=None,
exclude_cols=None
)
```

```
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3])
dataset.element_spec
TensorSpec(shape=(), dtype=tf.int32, name=None)
```

```
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3])
```

```
dataset.element_spec
```

```
TensorSpec(shape=(), dtype=tf.int32, name=None)
```

```
apply(
transformation_func
) -> 'DatasetV2'
```


```
apply(
transformation_func
) -> 'DatasetV2'
```


Documentation

A Dataset comprising lines from one or more CSV files.

Inherits From: Dataset, Dataset

Attributes

For more information,
read this guide.

Methods

View source

Applies a transformation function to this dataset.

apply enables chaining of custom Dataset transformations, which are represented as functions that take one Dataset argument and return a transformed Dataset.

as_numpy_iterator

View source

Returns an iterator which converts all elements of the dataset to numpy.

Use as_numpy_iterator to inspect the content of your dataset. To see element shapes and types, print dataset elements directly instead of using as_numpy_iterator.

This method requires that you are running in eager mode and the dataset's element_spec contains only TensorSpec components.

as_numpy_iterator() will preserve the nested structure of dataset elements.

View source

Combines consecutive elements of this dataset into batches.

The components of the resulting element will have an additional outer dimension, which will be batch_size (or N % batch_size for the last element if batch_size does not divide the number of input elements N evenly and drop_remainder is False). If your program depends on the batches having the same outer dimension, you should set the drop_remainder argument to True to prevent the smaller batch from being produced.

bucket_by_sequence_length

View source

A transformation that buckets elements in a Dataset by length.

Elements of the Dataset are grouped together by length and then are padded and batched.

This is useful for sequence tasks in which the elements have variable length. Grouping together elements that have similar lengths reduces the total fraction of padding in a batch which increases training step efficiency.

Below is an example to bucketize the input data to the 3 buckets "[0, 3), [3, 5), [5, inf)" based on sequence length, with batch size 2.

View source

Caches the elements in this dataset.

The first time the dataset is iterated over, its elements will be cached either in the specified file or in memory. Subsequent iterations will use the cached data.

When caching to a file, the cached data will persist across runs. Even the first iteration through the data will read from the cache file. Changing the input pipeline before the call to .cache() will have no effect until the cache file is removed or the filename is changed.

cardinality

View source

Returns the cardinality of the dataset, if known.

cardinality may return tf.data.INFINITE_CARDINALITY if the dataset contains an infinite number of elements or tf.data.UNKNOWN_CARDINALITY if the analysis fails to determine the number of elements in the dataset (e.g. when the dataset source is a file).

choose_from_datasets

View source

Creates a dataset that deterministically chooses elements from datasets.

For example, given the following datasets:

The elements of result will be:

concatenate

View source

Creates a Dataset by concatenating the given dataset with this dataset.

counter

View source

Creates a Dataset that counts from start in steps of size step.

Unlike tf.data.Dataset.range, which stops at some ending number, tf.data.Dataset.counter produces elements indefinitely.

enumerate

View source

Enumerates the elements of this dataset.

It is similar to python's enumerate.

filter

View source

Filters this dataset according to predicate.

filter_with_legacy_function

View source

Filters this dataset according to predicate. (deprecated)

fingerprint

View source

Computes the fingerprint of this Dataset.

If two datasets have the same fingerprint, it is guaranteeed that they would produce identical elements as long as the content of the upstream input files does not change and they produce data deterministically.

However, two datasets producing identical values does not always mean they would have the same fingerprint due to different graph constructs.

In other words, if two datasets have different fingerprints, they could still produce identical values.

flat_map

View source

Maps map_func across this dataset and flattens the result.

The type signature is:

Use flat_map if you want to make sure that the order of your dataset stays the same. For example, to flatten a dataset of batches into a dataset of their elements:

tf.data.Dataset.interleave() is a generalization of flat_map, since flat_map produces the same output as tf.data.Dataset.interleave(cycle_length=1)

from_generator

View source

Creates a Dataset whose elements are generated by generator. (deprecated arguments)

The generator argument must be a callable object that returns an object that supports the iter() protocol (e.g. a generator function).

The elements generated by generator must be compatible with either the given output_signature argument or with the given output_types and (optionally) output_shapes arguments, whichever was specified.

The recommended way to call from_generator is to use the

output_signature argument. In this case the output will be assumed to consist of objects with the classes, shapes and types defined by tf.TypeSpec objects from output_signature argument:

There is also a deprecated way to call from_generator by either with output_types argument alone or together with output_shapes argument. In this case the output of the function will be assumed to consist of tf.Tensor objects with the types defined by output_types and with the shapes which are either unknown or defined by output_shapes.

from_sparse_tensor_slices

View source

Splits each rank-N tf.sparse.SparseTensor in this dataset row-wise. (deprecated)

from_tensor_slices

View source

Creates a Dataset whose elements are slices of the given tensors.

The given tensors are sliced along their first dimension. This operation preserves the structure of the input tensors, removing the first dimension of each tensor and using it as the dataset dimension. All input tensors must have the same size in their first